**Start: select project execution path**
A or B

## A. Pangenome / DNABERT

**Load graph and model settings**
GFA; optional indexed labels
Pretrained or trained checkpoint

**Prepare graph-derived sequences**
Parse -> directed graph -> DAG
Extract paths -> concatenate DNA

**Encode DNA for DNABERT**
Checkpoint-aligned k-mer IDs
Special tokens, masks and padding

**Training mode: fit and validate**
Group inputs; train / val / test split
Fine-tune; select validation threshold

**Predict with a trained model**
Training run: held-out test inputs
Inference run: new encoded inputs

**Evaluate when labels are available**
Keep test data out of model selection
Fixed-threshold metrics

**Present and preserve results**
Class probabilities and predictions
Training: checkpoint + experiment.json

## B. FASTA / classical ML

**Load and audit FASTA dataset**
APP / PSEN1 / PSEN2 + folder labels
Inspect duplicates and label conflicts

**Separate training and test data**
Gene + class-stratified holdout
Five folds within training partition

**Fit candidate pipelines in CV**
Training-fold references and features
Scaling; optional selection / SMOTE

**Choose and evaluate the winner**
Select by mean CV macro F1; refit
Evaluate once on reserved holdout

**Keep fitted pipeline for web use**
Flask stores the trained pipeline
Dashboard shows training results

**Validate and predict new upload**
FASTA + trained gene; input checks
Reuse fitted transforms and classifier

**Return results to the user**
Per-record class and probabilities
Dashboard metrics and visualizations

Bypass route: inference-only mode loads a trained checkpoint and its fixed threshold.

**A and B are two execution paths within one project.**